Human Preferences as Verifiable Rewards for GRPO

Motivation

Reinforcement learning from verifiable rewards (RLVR) is strongest when a completion can be checked automatically: a math answer, a unit test result, a code execution trace, a tool outcome, or a symbolic solution. GRPO is a practical policy-optimization method for this setting [1]. This leaves a gap for open-ended assistant behavior, where the supervision often comes as human preferences rather than a single correct answer.

Poster claim. Human rankings can be turned into unit-test-like reward tasks: the target is a permutation, the checker is deterministic, and the training signal still carries human preference information.

Core Idea

For each user request x, annotators provide an ordered list of candidate answers, as in preference-feedback datasets and evaluation pipelines [2]:

$$y_1^\star \succ y_2^\star \succ \cdots \succ y_m^\star.$$

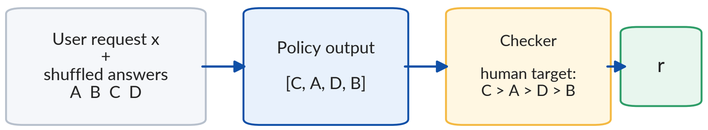
During training, the candidates are shuffled and the policy must output the correct ranking as a permutation of answer IDs.

$$\text{prompt: } (x, \{y_1, \ldots, y_m\}) \quad \longrightarrow \quad \text{output: } [i_1, i_2, \ldots, i_m].$$

For ranking prompts, the scalar reward is deterministic because the assessor order is known. A useful version gives partial credit for pairwise comparisons and stronger credit for the complete order:

$$r = \lambda_1 \,\mathrm{PairAcc} + \lambda_2 \,\mathrm{NDCG} + \lambda_3 \,\mathrm{ExactMatch}.$$

The task is verifiable even though the original supervision is human: after annotation, every training sample has a fixed, automatically checkable answer.



Preference annotation is transformed into a deterministic permutation-checking task.

Why This Is Different

The model is not asked to imitate one preferred answer. It must compare several plausible answers to the same request and recover the full best-to-worst ordering. This makes the model practice *judgment*, not only answer production.

- **Standard RLVR:** verifies task outcomes: math answers, code tests, tool results, and other automatic checks.
- **Ranking RLVR:** verifies response-quality judgment.
- **Mixed RLVR:** trains task solving and judgment in one policy.

Training Formulation

Ranking prompts are added to the same GRPO/RLVR stream as ordinary verifiable tasks. These base tasks can be math, code, tool-use, structured QA, or any task with an automatic checker. The training distribution becomes:

$$\mathcal{D}_{\text{train}} = \mathcal{D}_{\text{base}} \cup \mathcal{D}_{\text{rank}}.$$

The ranking task therefore fits the same loop as other RLVR tasks: sample, parse, check, normalize, update.

On-policy GRPO

$$y_i \sim \pi_\theta(\cdot \mid x), \qquad A_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$$

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G} A_i \nabla_\theta \log \pi_\theta(y_i \mid x)\right].$$

Here $\bar{r}$ is the group mean and σ_r the group std of rewards. There is no clipping or old-policy importance ratio: the same on-policy samples define the update.

Prompt Contract

Input: one user request and m candidate assistant answers in randomized order.
Output: only the answer IDs sorted from best to worst.
Reward: exact and partial credit against the hidden assessor ranking.

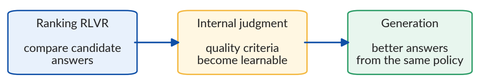
Checker

- Parse only a compact permutation, e.g. `[C, A, D, B]`.
- Penalize invalid IDs, duplicate IDs, missing IDs, and extra text.
- Score all candidate pairs so the reward remains informative when the full order is not recovered.

Transfer Hypothesis

The central hypothesis is that judgment transfers to generation. By learning to distinguish stronger answers from weaker ones, the policy internalizes quality features that are useful when it later writes its own answer.

- **Contrastive signal.** The model sees multiple plausible answers for one request, so it must identify what separates good from merely fluent.
- **Preference awareness.** The task exposes latent criteria such as helpfulness, factuality, completeness, instruction following, tone, and safety.
- **Judge-to-generator bridge.** This follows the broader line of work where models first learn to evaluate responses and then use that ability to improve generation [3, 4, 5].

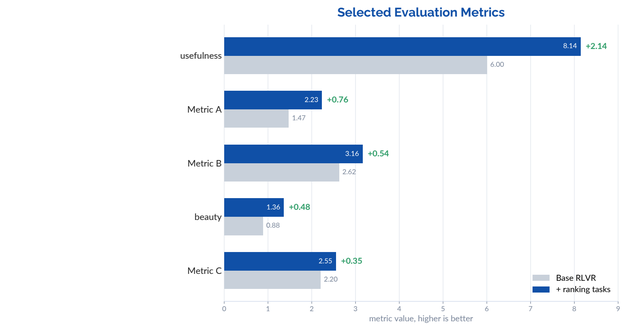


Experimental Readout

The experiments compare a standard RLVR baseline with a mixed setup where ranking tasks are added to the RLVR/GRPO training stream.

Setting	Verifiable task	Main purpose
Standard RLVR	automatic task check	capability anchor
Ranking-only	assessor order check	judgment skill
Mixed RLVR	base tasks + ranking	transfer to generation

Main observation. In the mixed RLVR experiments, adding ranking tasks improved metrics tied to reward models and RLHF-style answer quality, while preserving the benefits of automatically checked RLVR tasks.



Design Lessons

- Use partial-credit ranking rewards; exact-match reward alone is too sparse for long candidate lists.
- Keep base verifiable tasks in the mix to preserve hard capability pressure from math, code, tool-use, and other checked domains.
- Hold out requests, not only candidate permutations, to measure genuine generalization.

Takeaway

Human rankings can be reused as verifiable reward tasks. This creates a practical bridge between preference annotation and RLVR: the model learns to judge answer quality, and that judgment can transfer back into better generation.

References

[1] Zhihong Shao et al. “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models”. In: *arXiv preprint arXiv:2402.03300* (2024).

[2] Ganqu Cui et al. “UltraFeedback: Boosting Language Models with Scaled AI Feedback”. In: *arXiv preprint arXiv:2310.01377* (2023).

[3] Lianghui Zhu, Xinggang Wang, and Xinlong Wang. “JudgeLM: Fine-Tuned Large Language Models are Scalable Judges”. In: *arXiv preprint arXiv:2310.17631* (2023).

[4] Weizhe Yuan et al. “Self-Rewarding Language Models”. In: *arXiv preprint arXiv:2401.10020* (2024).

[5] Jiangshan Duo et al. “JudgeRLVR: Judge First, Generate Second for Efficient Reasoning”. In: *arXiv preprint arXiv:2601.08468* (2026).